



**UNIVERSIDAD NACIONAL DEL CENTRO DEL PERU**  
**FACULTAD DE INGENIERIA DE SISTEMAS**  
**DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS**  
**PROGRAMA DE INGENIERÍA DE SISTEMAS**



**Práctica SEMANA 11**

**Asignatura:** Desarrollo de Aplicaciones Web (IS093A)

**Unidad II:** Desarrollo Web FullStack

**Tema:** Introducción a Django: Arquitectura MTV, URLs, Vistas (FBV/CBV), Plantillas (Herencia, Bloques, Tags, Filtros) y Modelos (ORM, Consultas)

**Modalidad:** Presencial / Laboratorio

**Fecha:** 17.06.2026

**APELLIDOS Y NOMBRES:** Barja Ortiz Erick Gerson

**OBJETIVO DE LA PRÁCTICA**

Construir una aplicación web funcional con Django 5.x, aplicando el patrón MTV (Model-Template-View), configurando enrutamiento con `urls.py`, desarrollando vistas basadas en funciones (FBV) y clases (CBV), implementando plantillas con herencia, bloques, etiquetas y filtros, y definiendo modelos con el ORM de Django para persistencia en SQLite. Validar el ciclo solicitud-respuesta, la migración de base de datos y preparar la estructura para formularios, admin y sesiones (Semana 12).

**RECURSOS REQUERIDOS**

- Python 3.11+ + pip
- Visual Studio Code (Extensiones: Python, Django, SQLite Viewer, Pylint)
- Terminal integrada + entorno virtual (venv)
- Navegador moderno + DevTools
- Documentación oficial: [docs.djangoproject.com/en/5.0/](https://docs.djangoproject.com/en/5.0/)
- Conexión a internet + GitHub Hub

**EJERCICIO PASO A PASO**

1. Crear entorno virtual, instalar Django, iniciar proyecto (startproject) y app (startapp).

Verificar runserver y estructura de carpetas

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Microsoft Windows [Versión 10.0.19041.264]
(c) 2020 Microsoft Corporation. Todos los derechos reservados.

D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>python -m venv venv

D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>venv\Scripts\Activate.ps1

D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>pip install "django>=5.0,<5.1"
Collecting django<5.1,>=5.0
  Downloading Django-5.0.14-py3-none-any.whl.metadata (4.1 kB)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

46.1/46.1 kB 2.4 MB/s eta 0:00:00
Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
349.3/349.3 kB 803.5 kB/s eta 0:00:00
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.11.1 django-5.0.14 sqlparse-0.5.5 tzdata-2026.2

[notice] A new release of pip is available: 24.0 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip

D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

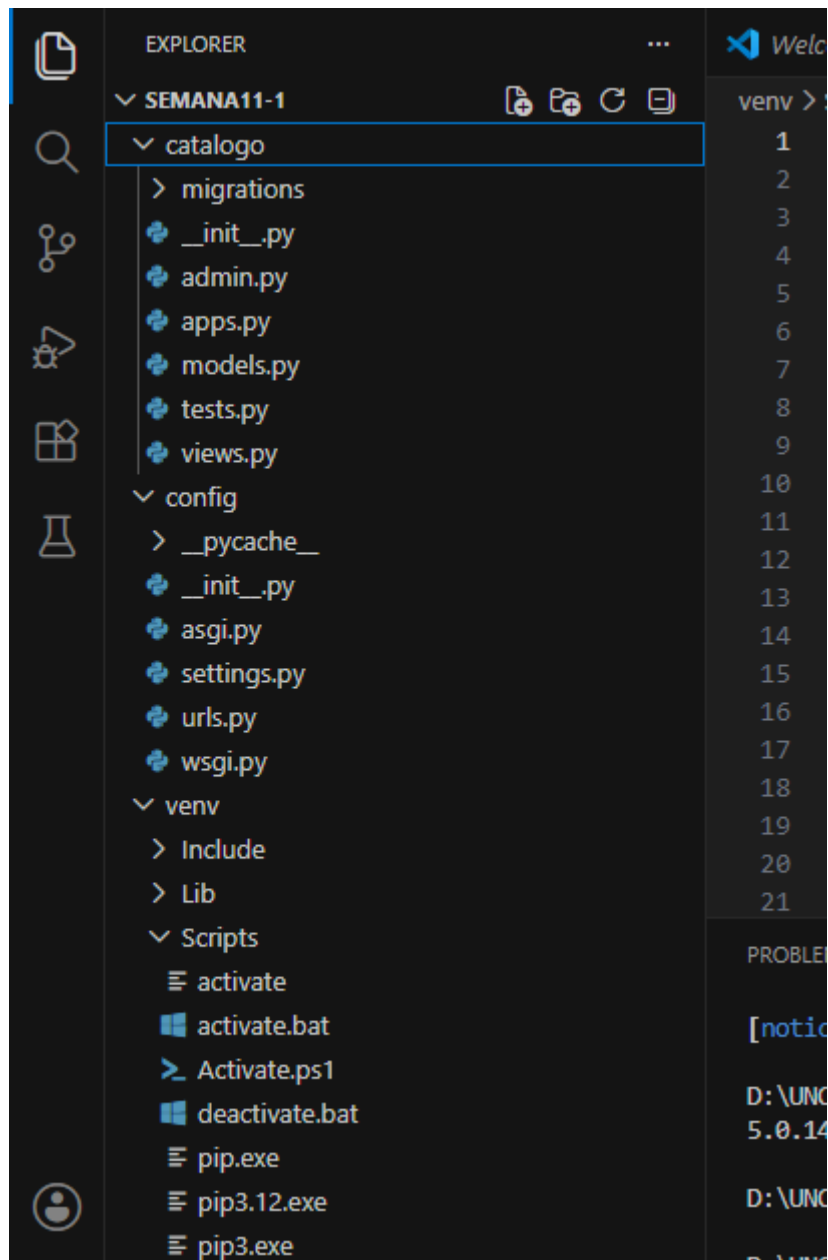
[notice] To update, run: python.exe -m pip install --upgrade pip

D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>python -m django --version
5.0.14

D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>django-admin startproject config .

D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>python manage.py startapp catalogo

D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>
```

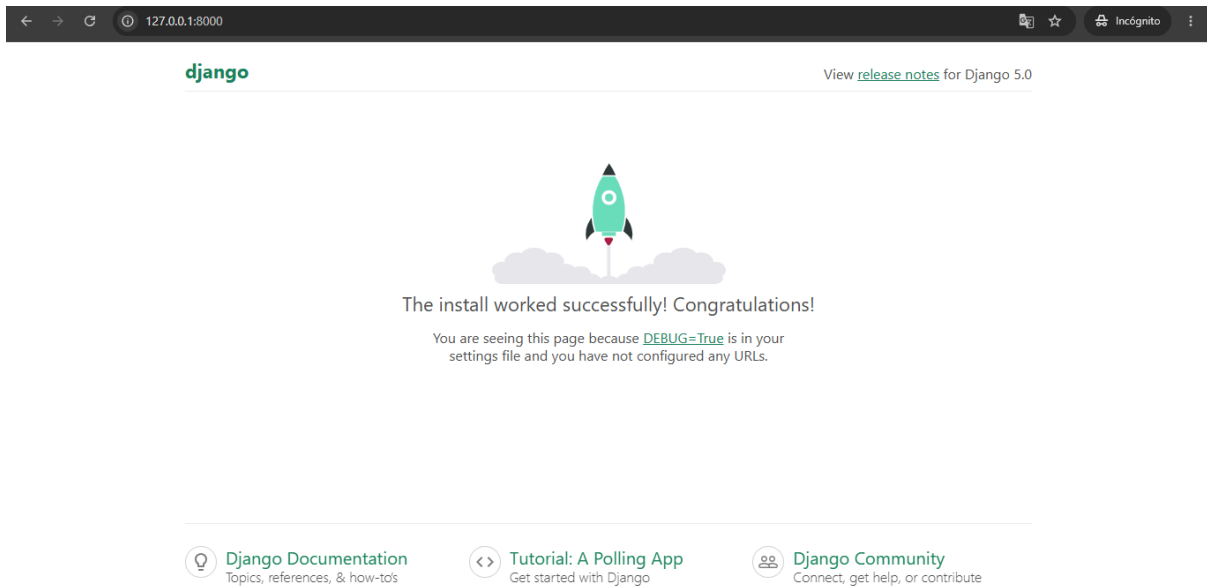


```
... Activate.ps1 settings.py
config > settings.py > ...
28 ALLOWED_HOSTS = []
29
30
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'catalogo',
41 ]
42
43 MIDDLEWARE = [
44     'django.middleware.security.SecurityMiddleware',
45     'django.contrib.sessions.middleware.SessionMiddleware',
46     'django.middleware.common.CommonMiddleware',
47     'django.middleware.csrf.CsrfViewMiddleware',
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run 'python manage.py migrate' to apply them.
June 17, 2026 - 13:24:41
Django version 5.0.14, using settings 'config.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

[17/Jun/2026 13:25:05] "GET / HTTP/1.1" 200 10629
Not Found: /favicon.ico
[17/Jun/2026 13:25:06] "GET /favicon.ico HTTP/1.1" 404 2110
```



2. Configurar rutas: `app/urls.py`, incluirla en `project/urls.py`. Mapear `path()` a vistas. Usar `name` para reversibilidad.

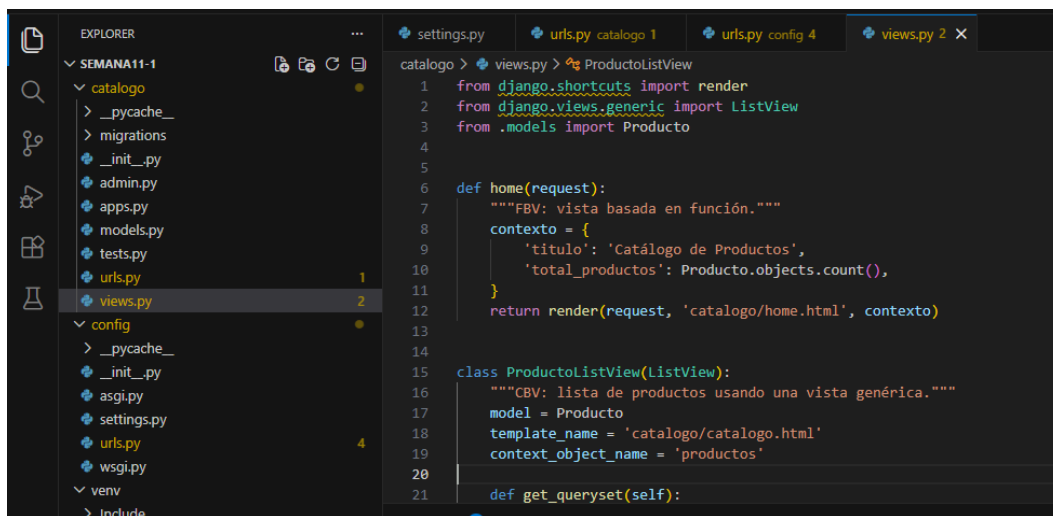
The screenshot shows a code editor with the Explorer pane on the left displaying the project structure. The 'urls.py' file for the 'catalogo' app is selected. The main editor shows the following code:

```
1 from django.urls import path
2 from . import views
3
4 app_name = 'catalogo'
5
6 urlpatterns = [
7     path('', views.home, name='home'),
8     path('productos/', views.ProductoListView.as_view(), name='producto_list'),
9 ]
```

The screenshot shows the same code editor with the 'urls.py' file for the 'catalogo' app. The code has been updated to include the 'include()' function and the admin site URLs. The code is as follows:

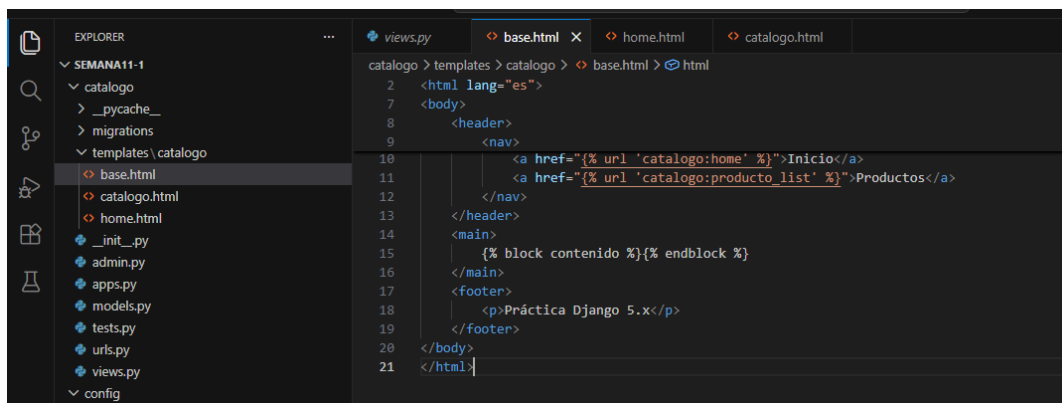
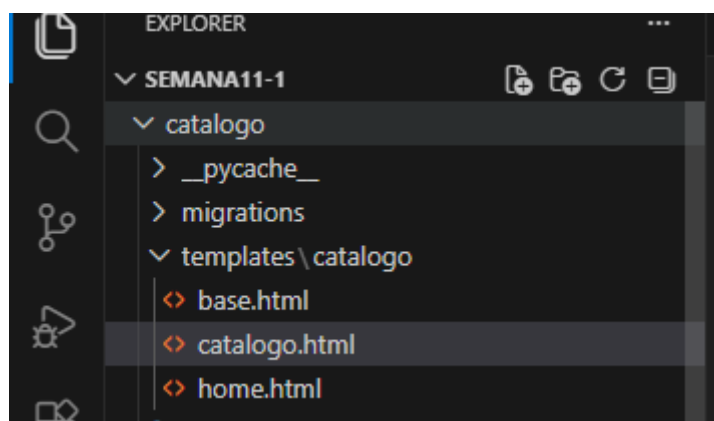
```
9 2. Add a URL to urlpatterns: path('', views.home, name='home')
10
11 Class-based views
12 1. Add an import: from other_app.views import Home
13 2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
14 Including another URLconf
15 1. Import the include() function: from django.urls import include, path
16 2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
17
18 from django.contrib import admin
19 from django.urls import path
20
21 urlpatterns = [
22     path('admin/', admin.site.urls),
23     path('', include('catalogo.urls')),
24 ]
```

3. Implementar vistas: 1 FBV (def home) y 1 CBV (ListView). Retornar HttpResponse o render(). Pasar contexto.



```
1 from django.shortcuts import render
2 from django.views.generic import ListView
3 from .models import Producto
4
5
6 def home(request):
7     """FBV: vista basada en función."""
8     contexto = {
9         'titulo': 'Catálogo de Productos',
10        'total_productos': Producto.objects.count(),
11    }
12    return render(request, 'catalogo/home.html', contexto)
13
14
15 class ProductoListView(ListView):
16     """CBV: lista de productos usando una vista genérica."""
17     model = Producto
18     template_name = 'catalogo/catalogo.html'
19     context_object_name = 'productos'
20
21     def get_queryset(self):
```

4. Crear plantillas: base.html con bloques, catalogo.html con {% extends %}, {% for %}, {% if %}, filtros ('title, date').



```
1 <html lang="es">
2 <body>
3 <header>
4 <nav>
5 <a href="{% url 'catalogo:home' %}">Inicio</a>
6 <a href="{% url 'catalogo:producto_list' %}">Productos</a>
7 </nav>
8 </header>
9 <main>
10 {% block contenido %}{% endblock %}
11 </main>
12 <footer>
13 <p>Práctica Django 5.x</p>
14 </footer>
15 </body>
16 </html>
```

This screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The project structure includes a 'SEMANA11-1' folder with subfolders 'catalogo' and 'templates/catalogo'. The 'home.html' file is selected in the Explorer and opened in the editor. The breadcrumb navigation at the top of the editor shows the path: 'catalogo > templates > catalogo > home.html > ...'. The code in 'home.html' is a Django template that extends 'catalogo/base.html'. It contains two blocks: 'titulo' and 'contenido'. The 'contenido' block renders an h1 tag with the title and a paragraph showing the total number of products and a link to view them.

```
catalogo > templates > catalogo > home.html > ...
1  {% extends 'catalogo/base.html' %}
2
3  {% block titulo %}Inicio - {{ titulo }}{% endblock %}
4
5  {% block contenido %}
6      <h1>{{ titulo }}</h1>
7      <p>Tenemos {{ total_productos }} producto(s) registrados.</p>
8  {% endblock %}
```

This screenshot shows the Visual Studio Code editor with the 'catalogo.html' file selected in the Explorer and opened in the editor. The breadcrumb navigation shows the path: 'catalogo > templates > catalogo > catalogo.html > ...'. The code in 'catalogo.html' is a Django template that extends 'catalogo/base.html'. It contains two blocks: 'titulo' and 'contenido'. The 'contenido' block uses a conditional statement to check if there are products. If there are, it renders a list of products with their names, prices, and creation dates. If there are no products, it renders a message saying 'No hay productos disponibles'.

```
catalogo > templates > catalogo > catalogo.html > ...
1  {% extends 'catalogo/base.html' %}
2
3  {% block titulo %}{{ titulo }}{% endblock %}
4
5  {% block contenido %}
6      <h1>{{ titulo }}</h1>
7
8      {% if productos %}
9          <ul>
10             {% for producto in productos %}
11                 <li>
12                     <strong>{{ producto.nombre|title }}</strong> - ${{ producto.precio }}
13                     <br>
14                     <small>Agregado el {{ producto.fecha_creacion|date:"d/m/Y" }}</small>
15                 </li>
16             {% endfor %}
17          </ul>
18      {% else %}
19          <p>No hay productos disponibles.</p>
20      {% endif %}
21  {% endblock %}
```

5. Definir modelo Producto, ejecutar makemigrations y migrate. Poblar datos, consultar con objects.all() y filter().

This screenshot shows the Visual Studio Code editor with the 'models.py' file selected in the Explorer and opened in the editor. The breadcrumb navigation shows the path: 'catalogo > models.py > ...'. The code in 'models.py' defines a Django model named 'Producto'. It inherits from 'models.Model'. The model has five fields: 'nombre' (CharField), 'descripcion' (TextField), 'precio' (DecimalField), 'stock' (PositiveIntegerField), and 'disponible' (BooleanField). It also has a 'fecha\_creacion' field (DateTimeField) with 'auto\_now\_add=True'. The 'Meta' class defines the ordering as '[-fecha\_creacion]'. The '\_\_str\_\_' method returns the 'nombre' attribute.

```
catalogo > models.py > ...
1  from django.db import models
2
3  class Producto(models.Model):
4      nombre = models.CharField(max_length=120)
5      descripcion = models.TextField(blank=True)
6      precio = models.DecimalField(max_digits=8, decimal_places=2)
7      stock = models.PositiveIntegerField(default=0)
8      disponible = models.BooleanField(default=True)
9      fecha_creacion = models.DateTimeField(auto_now_add=True)
10
11      class Meta:
12          ordering = ['-fecha_creacion']
13
14      def __str__(self):
15          return self.nombre
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Location: D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1\venv\Lib\site-packages
Requires: asgiref, sqlparse, tzdata
Required-by:

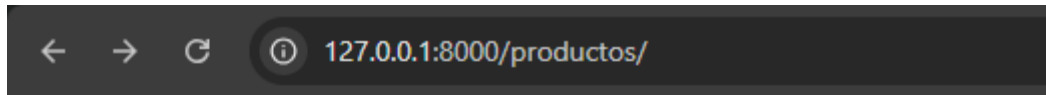
(venv) D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>

(venv) D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>python manage.py check
System check identified no issues (0 silenced).

(venv) D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
catalogo\migrations\0001_initial.py
- Create model Producto

(venv) D:\UNCP\IX SEMESTRE\Desarrollo de Aplicaciones Web\Semana11-1>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, catalogo, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
```



[Inicio Productos](#)

# Catálogo de Productos

No hay productos disponibles.

Práctica Django 5.x

Link en GitHub: <https://github.com/GErickOBTS/Semana11-1>